

251007-Application of Glasso on INT-Transformed Zero-Inflated Data

Jingxin Wang

2025-10-19

0. Last Time

1. Pearson partial correlation is more sensitive to transformation than Spearman and Kendall
2. Normality satisfied after INT transformation.
3. We can experiment Glasso on INT data, and compare the result with network given by Spearman and Kendall
4. Multiple test correction for p-value based edge selection should be tested.

1. Today

1. Application of Glasso on zero-inflated data before and after INT transformation
 - Compare the networks before and after transformations
2. Can we compare results of different transformation approaches with same Glasso rho?
 - How to put methylation before and after transformation into the same scale?
 - As rho increases, do paths of precision matrix elements change much before and after INT?
 - BIC trace curves: healthy vs cancers
 - Path of the precision matrix component
 - Can we center every column of the matrix and scale it by the same factor before Glasso?

```
## Load packages
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.2
## v ggplot2    4.0.0      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggplot2)
library(cowplot)
```

```
##
## Attaching package: 'cowplot'
##
## The following object is masked from 'package:lubridate':
##
##     stamp
```

```
library(igraph)
```

```
##
## Attaching package: 'igraph'
##
## The following objects are masked from 'package:lubridate':
##
##     %--%, union
##
## The following objects are masked from 'package:dplyr':
##
##     as_data_frame, groups, union
##
## The following objects are masked from 'package:purrr':
##
##     compose, simplify
##
## The following object is masked from 'package:tidyr':
##
##     crossing
##
## The following object is masked from 'package:tibble':
##
##     as_data_frame
##
## The following objects are masked from 'package:stats':
##
##     decompose, spectrum
##
## The following object is masked from 'package:base':
##
##     union
```

```
library(tidygraph)
```

```
##
## Attaching package: 'tidygraph'
##
## The following object is masked from 'package:igraph':
##
##     groups
```

```
##
## The following object is masked from 'package:stats':
##
##     filter
```

```
library(ggraph)
library(ppcor)
```

```
## Loading required package: MASS
##
## Attaching package: 'MASS'
##
## The following object is masked from 'package:tidygraph':
##
##     select
##
## The following object is masked from 'package:dplyr':
##
##     select
```

```
library(iDINGO)
library(HurdleNormal)
```

```
## Define Global functions
## Extract upper triangle part of matrices
```

```
takeUpperTriangle <- function(Mat) {
  Vec <- Mat[upper.tri(Mat)]
  as.vector(Vec)
}
```

```
## Compare the partial correlations before and after transformations for all types of correlations
```

```
compareParScatter <- function(refPar, expPar, title) {
  ggplot(data = data.frame("Before" = refPar, "After" = expPar)) +
    geom_point(aes(x = Before, y = After)) +
    coord_cartesian(xlim = c(-0.4,0.4), ylim = c(-0.4,0.4)) +
    geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey") +
    labs(title = title)
}
```

```
## Inverse Component Trajectory and eBIC path
```

```
precMatPath <- function(X, rho.Vec, transName) {
  p <- ncol(X); n <- nrow(X)

  upperTriMat <- matrix(nrow = 0, ncol = p*(p-1)/2)
  eBIC.vec <- c()
  for (rho in rho.Vec) {
    glassoMat <- glassoPreMat(original.X, rho = rho)
    upperTriVec <- takeUpperTriangle(glassoMat$wi)
    upperTriMat <- rbind(upperTriMat, upperTriVec)
    eBIC <- extendedBIC(0.5, glassoMat$wi, cov(original.X), n)
    eBIC.vec <- c(eBIC.vec, eBIC)
  }
}
```

```

par(mfrow = c(1,2))
matplot(x = rho.Vec, y = upperTriMat, type = "l", lty = 1,
        xlab = "rho", ylab = "Precision Matrix Coef",
        ylim = c(-0.3, 0.3), main = paste("Precision matrix element path of", transName))

plot(x = rho.Vec, y = eBIC.vec, type = "l", lty = 1,
     xlab = "rho", ylab = "Extended BIC",
     main = paste("Extended BIC of", transName))
}

```

2. Loading Data and Packages

```

## Load customized functions
source("../R-scripts/INT.R")

## Load and Sample Methylation Data
X.T <- readRDS(file = "../Data/Common_pan_cancer_hyper_bins_adjusted_and_normalized_cnt_in_SE.RDS")
set.seed(226)
sampDMR <- sample(rownames(X.T), 10) ## Sample only 10 DMRs for speed sake
original.X <- t(X.T[sampDMR, ])

## Load Cancer Types
sampleInfo <- read.csv("../Data/Common_pan_cancer_hyper_bins_adjusted_cnt_in_SE_samples_info.csv")
cancerTypes <- sampleInfo$cancer_type

## INT Transformation
INT.X <- INT(original.X, 0.25)

```

3. Glasso on Zero-Inflated the Data, Before and After INT, All Samples

3.1 Network Comparison

```

source("../R-scripts/graphMaker.R")
library(glasso)

# original.X <- scale(original.X, center = T, scale = F)/50 ## Centering and Scaling by a factor of 50

## Partial Pearson Correlation before any transformations
ParCorrNone <- pcor(original.X, method = "pearson")

## Partial Pearson Correlation after INT transformations
ParCorrINT <- pcor(INT.X, method = "pearson")

## Glasso before INT
glassoNone <- glassoPreMat(original.X, rho = 0.25)
ParCorrGlassoNone <- parCorr_from_precision(glassoNone$wi, 0.25, nrow(original.X))

## Glasso after IVT
glassoINT <- glassoPreMat(INT.X, rho = 0.25)

```

```

ParCorrGlassoINT <- parCorr_from_precision(glassoINT$wi, 0.25, nrow(INT.X))

## Graphical Comparison
ParCorrNoneGraph <- plot_parcorr_graph(ParCorrNone$estimate, NA, "All", "Before INT")
ParCorrINTGraph <- plot_parcorr_graph(ParCorrINT$estimate, NA, "All", "After INT")
ParCorrNoneGlassoGraph <- plot_parcorr_graph(ParCorrGlassoNone, 0.25, "All", "Before INT Glasso")
ParCorrINTGlassoGraph <- plot_parcorr_graph(ParCorrGlassoINT, 0.25, "All", "After INT Glasso")

plot_grid(plotlist = list(ParCorrNoneGraph, ParCorrINTGraph,
                          ParCorrNoneGlassoGraph, ParCorrINTGlassoGraph),
          nrow = 2, ncol = 2)

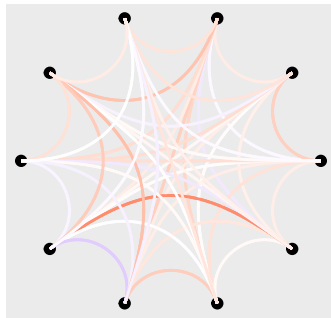
```

```

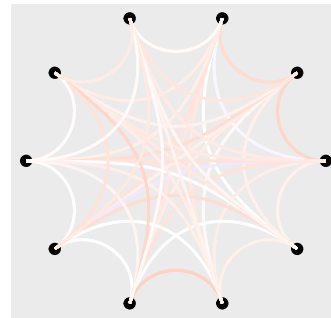
## Ignoring unknown labels:
## * colour : "Pcorr"
## Ignoring unknown labels:
## * colour : "Pcorr"
## Ignoring unknown labels:
## * colour : "Pcorr"
## Ignoring unknown labels:
## * colour : "Pcorr"

```

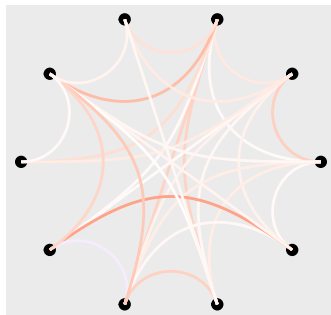
Before INT
rho = NA , All



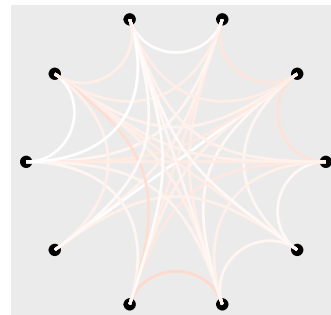
After INT
rho = NA , All



Before INT Glasso
rho = 0.25 , All



After INT Glasso
rho = 0.25 , All



```

ggsave(' ../ ../Images/GraphVis/AllSampleINTGlasso.png')

```

```

## Saving 6.5 x 4.5 in image

```

3.2 Does INT significantly alter precision matrix components?

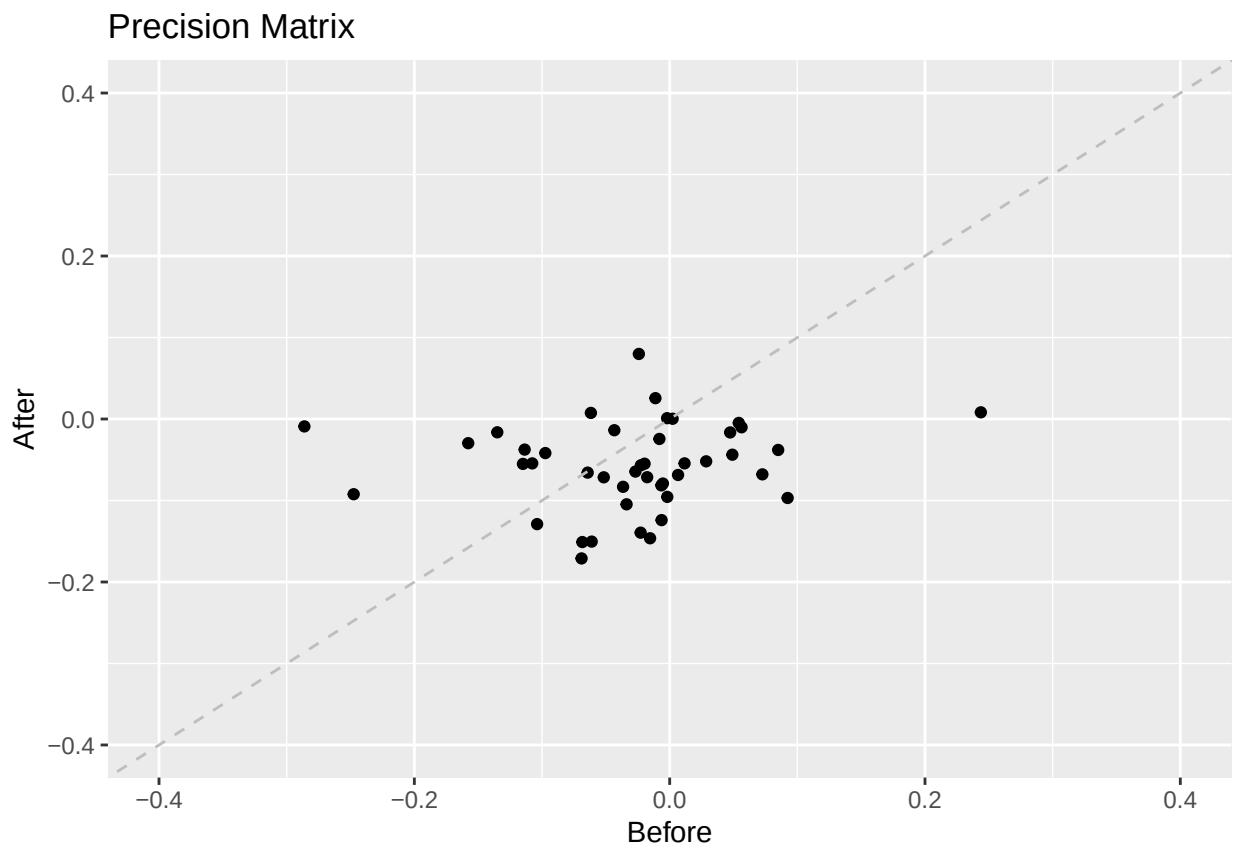
Next, we will use scatter plot to compare the strength of upper triangular components of precision matrices before and after INT transformation.

```
## Precision Matrix without penalty
condcenter.X <- conditionalCenter(original.X)
originalPrecMat <- solve(cov(condcenter.X))
intPrecMat <- solve(cov(INT.X))

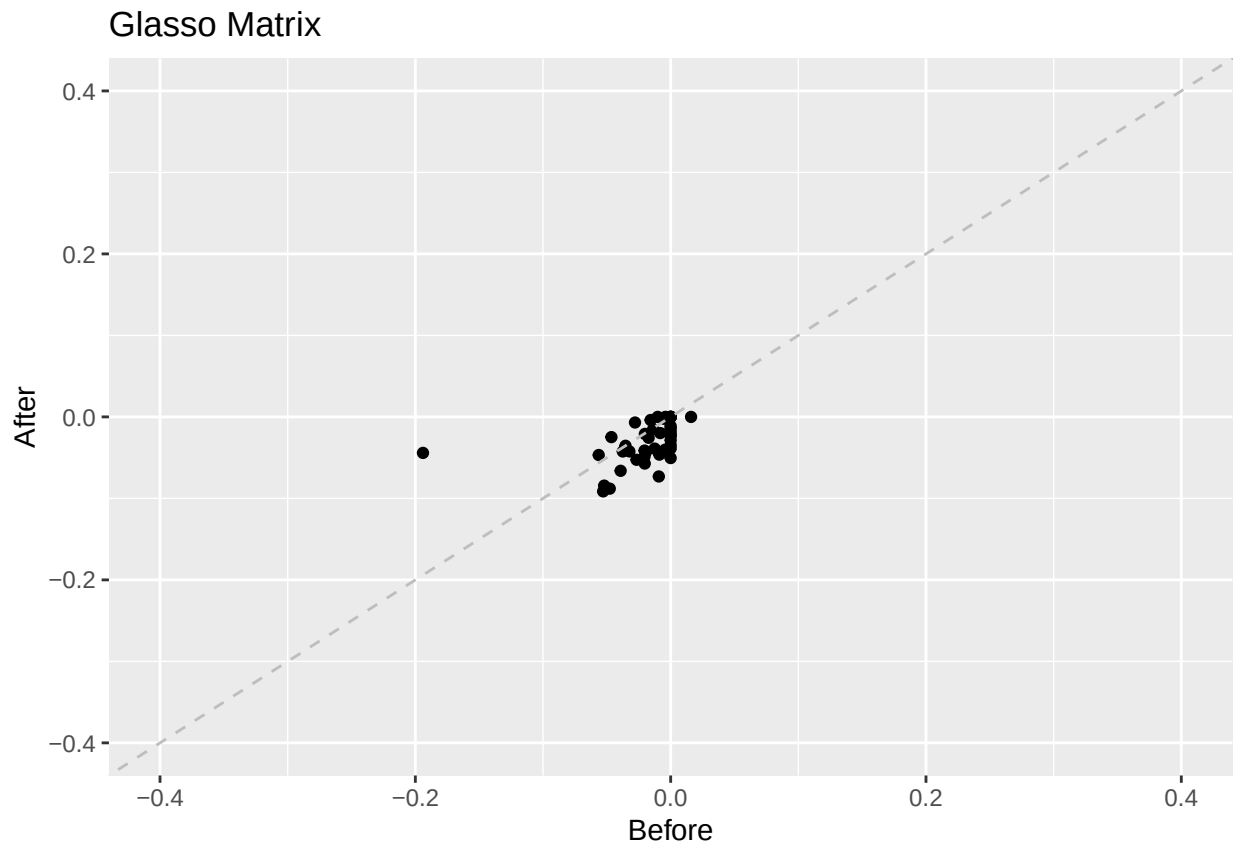
originalCompo <- takeUpperTriangle(originalPrecMat)
intCompo <- takeUpperTriangle(intPrecMat)

## Precision Matrix with penalty
originalGlasso <- takeUpperTriangle(glassoNone$wi)
intGlasso <- takeUpperTriangle(glassoINT$wi)

## Plotting
compareParScatter(originalCompo, intCompo, "Precision Matrix")
```



```
compareParScatter(originalGlasso, intGlasso, "Glasso Matrix")
```



```
## Which component deviated from the original precision the most?
cat("For unpenalized precision matrix -----\n")
```

```
## For unpenalized precision matrix -----
```

```
cat("The ", which.max(abs(originalCompo - intCompo)), "th component shows the biggest error of ", max(abs(originalCompo - intCompo)), "\n")
```

```
## The 19 th component shows the biggest error of 0.3365909
```

```
cat("As compared maximum absolute value of all components before transformation:", max(abs(originalCompo)), "\n")
```

```
## As compared maximum absolute value of all components before transformation: 0.4680884
```

```
cat("For glasso precision matrix -----\n")
```

```
## For glasso precision matrix -----
```

```
cat("The ", which.max(abs(originalGlasso - intGlasso)), "th component shows the biggest error of ", max(abs(originalGlasso - intGlasso)), "\n")
```

```
## The 19 th component shows the biggest error of 0.1498343
```

```
cat("As compared maximum absolute value of all components before transformation:", max(abs(originalGlas
```

```
## As compared maximum absolute value of all components before transformation: 0.1940115
```

```
abs(originalPrecMat - intPrecMat) == max(abs(originalCompo - intCompo))
```

```
##          3403603 8042636 7557950 8978709 8513995 3385302 523070 5560287 2086490
## 3403603   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE
## 8042636   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE
## 7557950   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE
## 8978709   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   TRUE   FALSE
## 8513995   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE
## 3385302   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE
## 523070    FALSE   FALSE   FALSE   TRUE    FALSE   FALSE   FALSE   FALSE   FALSE
## 5560287   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE
## 2086490   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE
## 2065685   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE   FALSE
##          2065685
## 3403603   FALSE
## 8042636   FALSE
## 7557950   FALSE
## 8978709   FALSE
## 8513995   FALSE
## 3385302   FALSE
## 523070    FALSE
## 5560287   FALSE
## 2086490   FALSE
## 2065685   FALSE
```

```
cat("Before INT, the 19th correlation is:", cor(original.X[, "523070"], original.X[, "8978709"]))
```

```
## Before INT, the 19th correlation is: 0.6541862
```

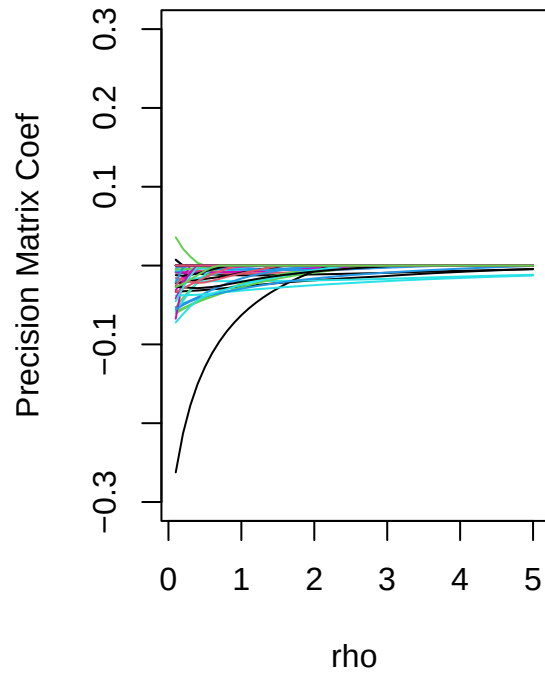
```
cat("After INT, the 19th correlation is:", cor(INT.X[, "523070"], INT.X[, "8978709"]))
```

```
## After INT, the 19th correlation is: 0.2695967
```

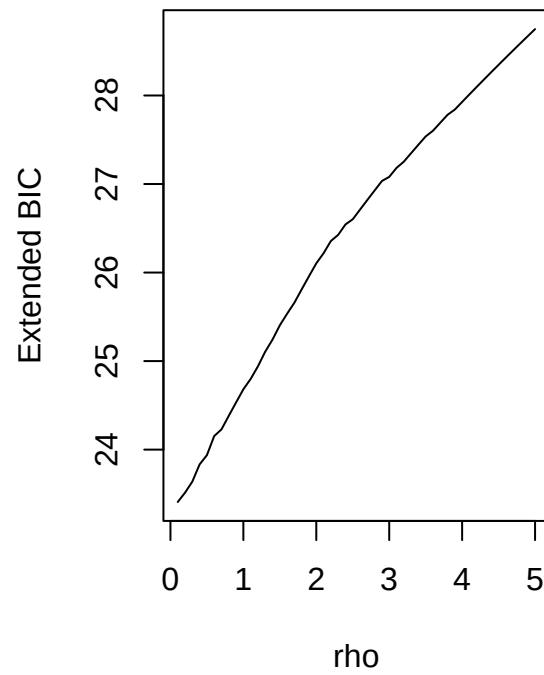


```
precMatPath(original.X, seq(0.1, 5, 0.1), "Original")
```

precision matrix element path of Ori

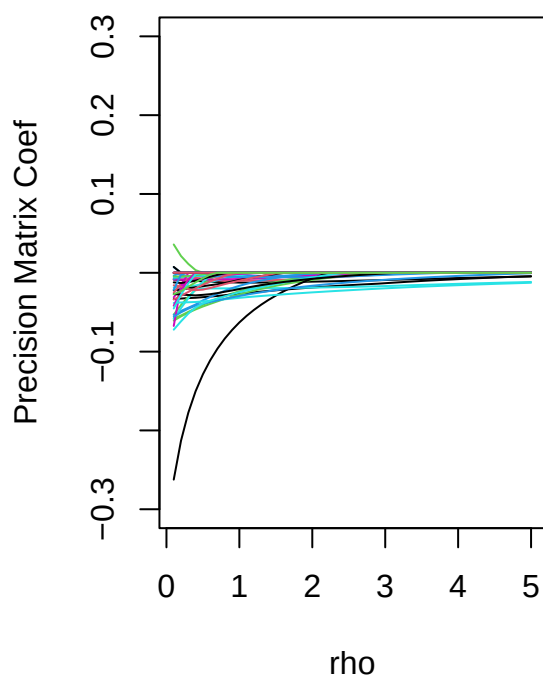


Extended BIC of Original

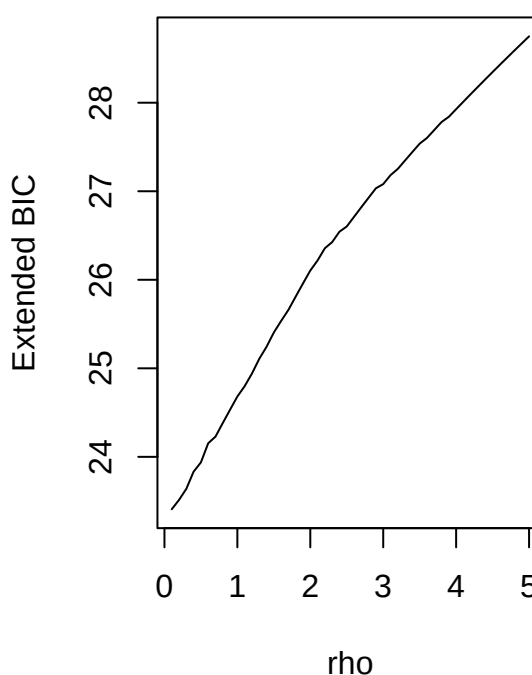


```
precMatPath(INT.X, seq(0.1, 5, 0.1), "INT")
```

Precision matrix element path of I



Extended BIC of INT



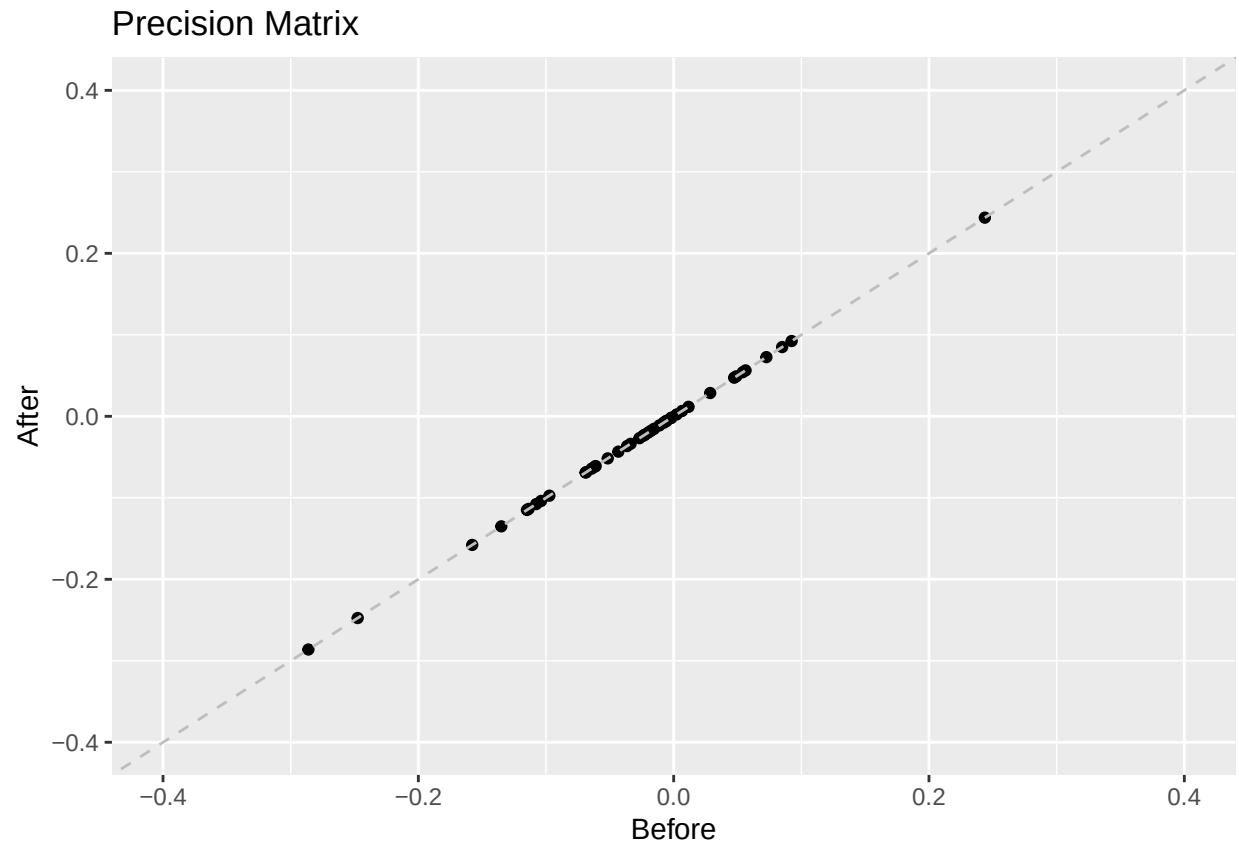
Main Takeaway: 1. INT transformation can lead to big differences in partial correlations from the original data. INT was not used in the right way or it simply distorted true partial correlation. 2. Convergence of all upper triangular components occur at $\rho = 2$

Observations: 1. For unpenalized Pcorr graphs, edge sets look similar before and after INT, but the color lightens after INT. 2. For glasso graphs, not only color lightens, but also removal of heavily weighted old edges and replacement by new and lighter weighted edges. 3. The error in inverse covariance matrix components before and after transformation can be very large relative to the size of the component before transformation. 4. The component giving the biggest error in the regular precision matrix also gives the biggest error in the glasso matrix; the component located at an off-diagonal entry. 5. There is very difference in the evolution path between original and INT-transformed glasso components. 6. As ρ increases, eBIC increases without any sign of slowing the increase. This could imply a worsening representation of the original partial correlation structure as the model grows sparser.

3.3 Does Conditional Centering significantly alter precision matrix?

```
## Precision Matrix without penalty
condcenter.X <- conditionalCenter(original.X)
condCenterPrecMat <- solve(cov(condcenter.X))
condCenterCompo <- takeUpperTriangle(condCenterPrecMat)

## Plotting
compareParScatter(originalCompo, condCenterCompo, "Precision Matrix")
```



```
## Which component deviated from the original precision the most?
cat("For unpenalized precision matrix -----\n")
```

```
## For unpenalized precision matrix -----
```

```
cat("The ", which.max(abs(originalCompo - condCenterCompo)), "th component shows the biggest error of "
```

```
## The 1 th component shows the biggest error of 0
```

```
cat("As compared maximum absolute value of all components before transformation:", max(abs(originalCompo - condCenterCompo))
```

```
## As compared maximum absolute value of all components before transformation: 0.4680884
```

```
abs(originalPrecMat - condCenterPrecMat) == max(abs(originalCompo - condCenterPrecMat))
```

```
## Warning in originalCompo - condCenterPrecMat: longer object length is not a
## multiple of shorter object length
```

```
##          3403603 8042636 7557950 8978709 8513995 3385302 523070 5560287 2086490
## 3403603  FALSE  FALSE  FALSE  FALSE  FALSE  FALSE  FALSE  FALSE  FALSE
## 8042636  FALSE  FALSE  FALSE  FALSE  FALSE  FALSE  FALSE  FALSE  FALSE
## 7557950  FALSE  FALSE  FALSE  FALSE  FALSE  FALSE  FALSE  FALSE  FALSE
```

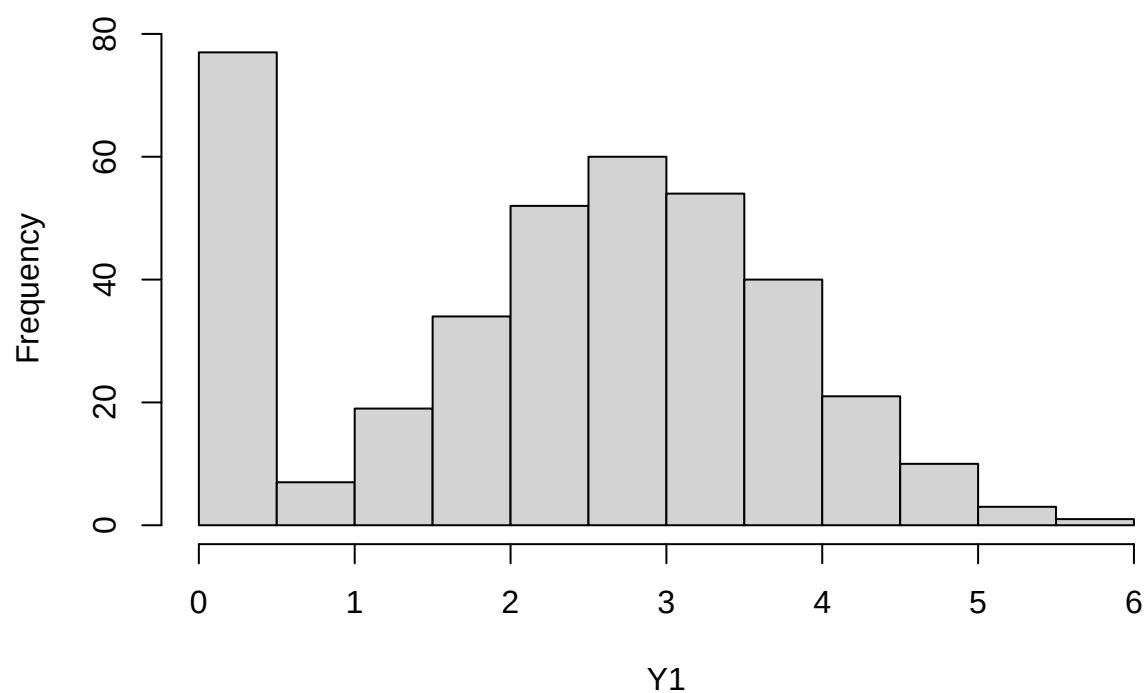
```
## 8978709 FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## 8513995 FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## 3385302 FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## 523070 FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## 5560287 FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## 2086490 FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## 2065685 FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##      2065685
## 3403603 FALSE
## 8042636 FALSE
## 7557950 FALSE
## 8978709 FALSE
## 8513995 FALSE
## 3385302 FALSE
## 523070 FALSE
## 5560287 FALSE
## 2086490 FALSE
## 2065685 FALSE
```

Conditional centering of non-zero values did not alter precision matrix components as much as INT.

4. HurdleNormal with Different the Transformation Methods: Original, INT+Repositioning Hurdle, Conditional Centering

```
## INT + Repositioning the Hurdle at 0
INT.X.reposit <- apply(INT.X, MARGIN = 2, FUN = function(x) x - min(x))
hist(INT.X.reposit[,1], main = "Marginal distribution after moving the hurdle to 0", xlab = "Y1")
```

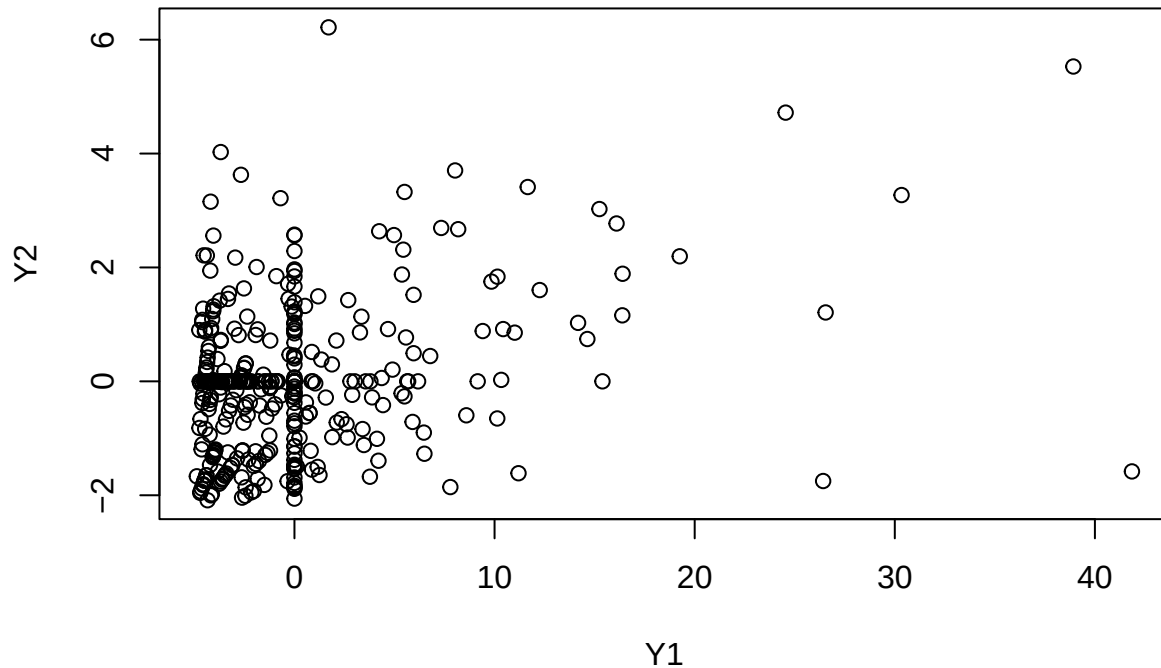
Marginal distribution after moving the hurdle to 0



Conditional centering will subtract the mean of non-zero values from original value.

```
condcenter.X <- conditionalCenter(original.X)
plot(condcenter.X[,1], condcenter.X[,3], main = "Joint Distribution of Y1, Y2 after condition centering")
```

Joint Distribution of Y1, Y2 after condition centering



```
hurdleComparison <- function(cancerType) {
  hurdleFit.original <- fitHurdle(original.X[cancerTypes == cancerType, ], nlambdas = 10)
  hurdleFit.INT <- fitHurdle(INT.X.reposit[cancerTypes == cancerType, ], nlambdas = 10)
  hurdleFit.condcenter <- fitHurdle(condcenter.X[cancerTypes == cancerType, ], nlambdas = 10)

  ## Compare path of extended BIC
  candidateModels <- list("Original" = hurdleFit.original,
                          "INT+Reposition" = hurdleFit.INT,
                          "Conditional Centering" = hurdleFit.condcenter)

  bicPlotList <- list()
  for (i in 1:3) {
    bicPlotList <- append(bicPlotList,
                          ggplot(data = candidateModels[[i]]$BIC_etc)+
                            geom_line(aes(x = trueEdges, y = BIC))+
                            labs(title = names(candidateModels)[i]))
  }

  plot_grid(plotlist = bicPlotList, nrow = 1) %>% print()

  ## Visualize the coef path
  coefPathList <- list()
  par(mfrow = c(1,3))
  for (i in 1:3) {
    coefPath <- lapply(candidateModels[[i]]$adjMat, FUN = takeUpperTriangle) %>%
      reduce(.f = rbind)
    rhoPath <- candidateModels[[i]]$lambda
  }
}
```

```

    matplot(x = rhoPath, y = coefPath, type = "l", lwd = 2,
            main = paste("Coef Paths:", names(candidateModels)[i]),
            xlab = "rho", ylab = "Off-diagonal Entries")
  }

  ## Network Comparison
  positList <- c(1,3,8,10)

  networkPlotList <- list()
  for (i in positList) {
    originalGraph <- plot_parcorr_graph(candidateModels[[1]]$adjMat[[i]] %>% as.matrix(),
                                         candidateModels[[1]]$lambda[i] %>% round(2),
                                         "Healthy", "Original")

    intGraph <- plot_parcorr_graph(candidateModels[[2]]$adjMat[[i]] %>% as.matrix(),
                                   candidateModels[[2]]$lambda[i] %>% round(2),
                                   "Healthy", "INT")

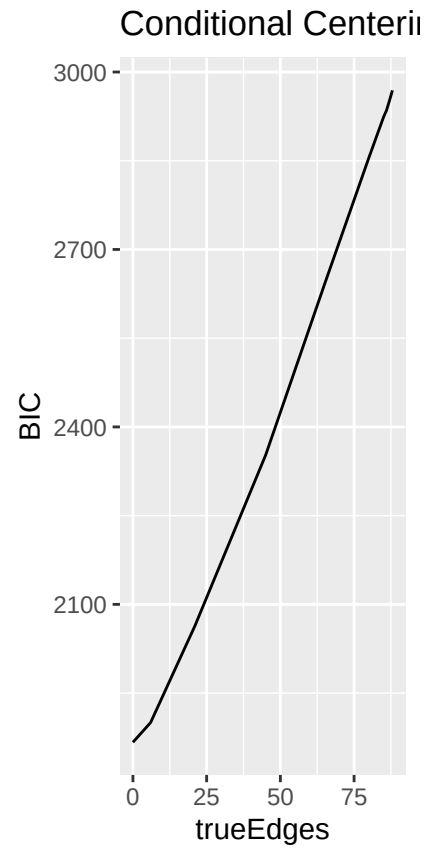
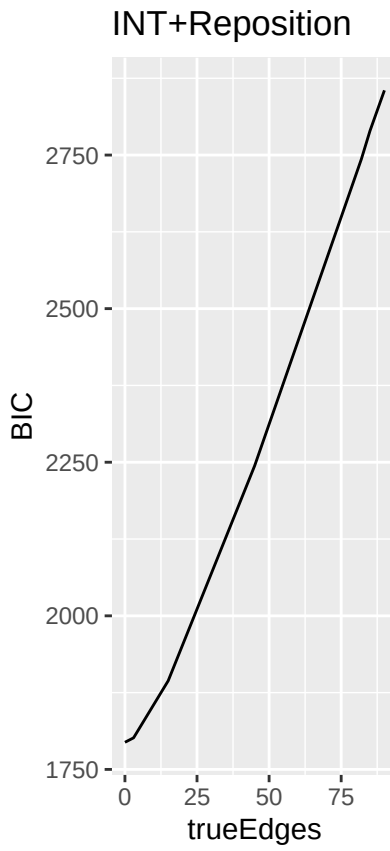
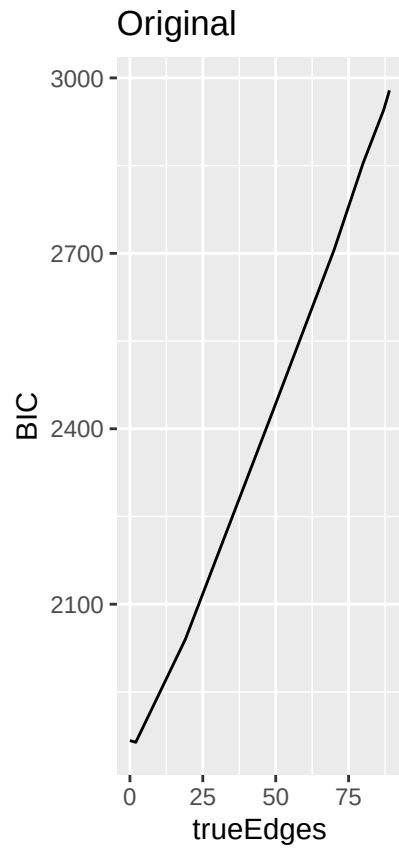
    condCenterGraph <- plot_parcorr_graph(candidateModels[[3]]$adjMat[[i]] %>% as.matrix(),
                                           candidateModels[[3]]$lambda[i] %>% round(2),
                                           "Healthy", "Conditional Centering")

    networkPlotList <- append(networkPlotList,
                              plot_grid(plotlist = list(originalGraph, intGraph, condCenterGraph),
                                      nrow = 1))
  }

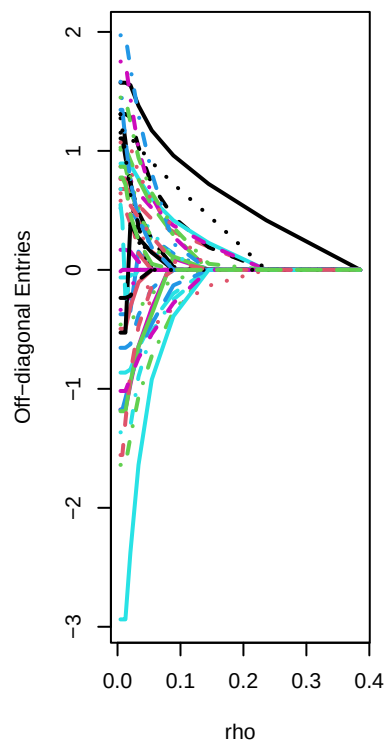
  plot_grid(plotlist = networkPlotList[1:2], ncol = 1, nrow = 4) %>% print()
  plot_grid(plotlist = networkPlotList[1:2], ncol = 1, nrow = 4) %>% print()
  gc()
  return(0)
}

hurdleComparison("Normal")

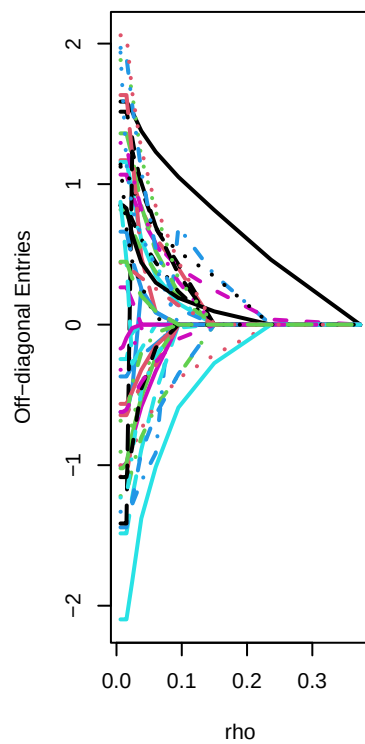
```



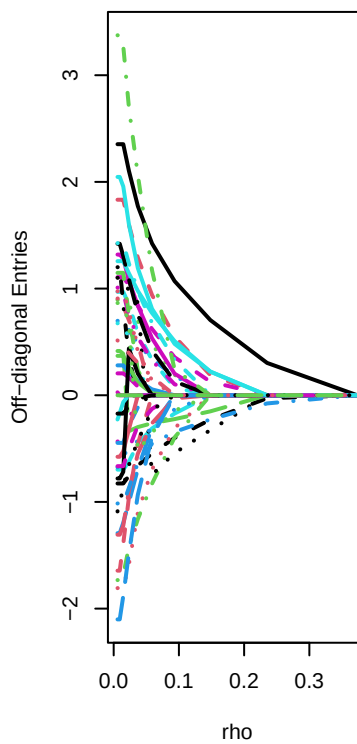
Coef Paths: Original



Coef Paths: INT+Reposition

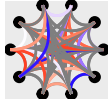


Coef Paths: Conditional Centeri



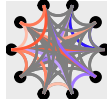
Original

$\rho = 0$, Healthy



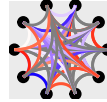
INT

$\rho = 0.01$, Healthy



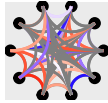
Conditional Cent

$\rho = 0.01$, Healthy



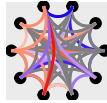
Original

$\rho = 0.01$, Healthy



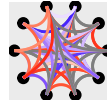
INT

$\rho = 0.02$, Healthy



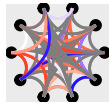
Conditional Cent

$\rho = 0.01$, Healthy



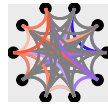
Original

$\rho = 0$, Healthy



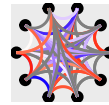
INT

$\rho = 0.01$, Healthy



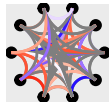
Conditional Cent

$\rho = 0.01$, Healthy



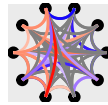
Original

$\rho = 0.01$, Healthy



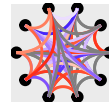
INT

$\rho = 0.02$, Healthy



Conditional Cent

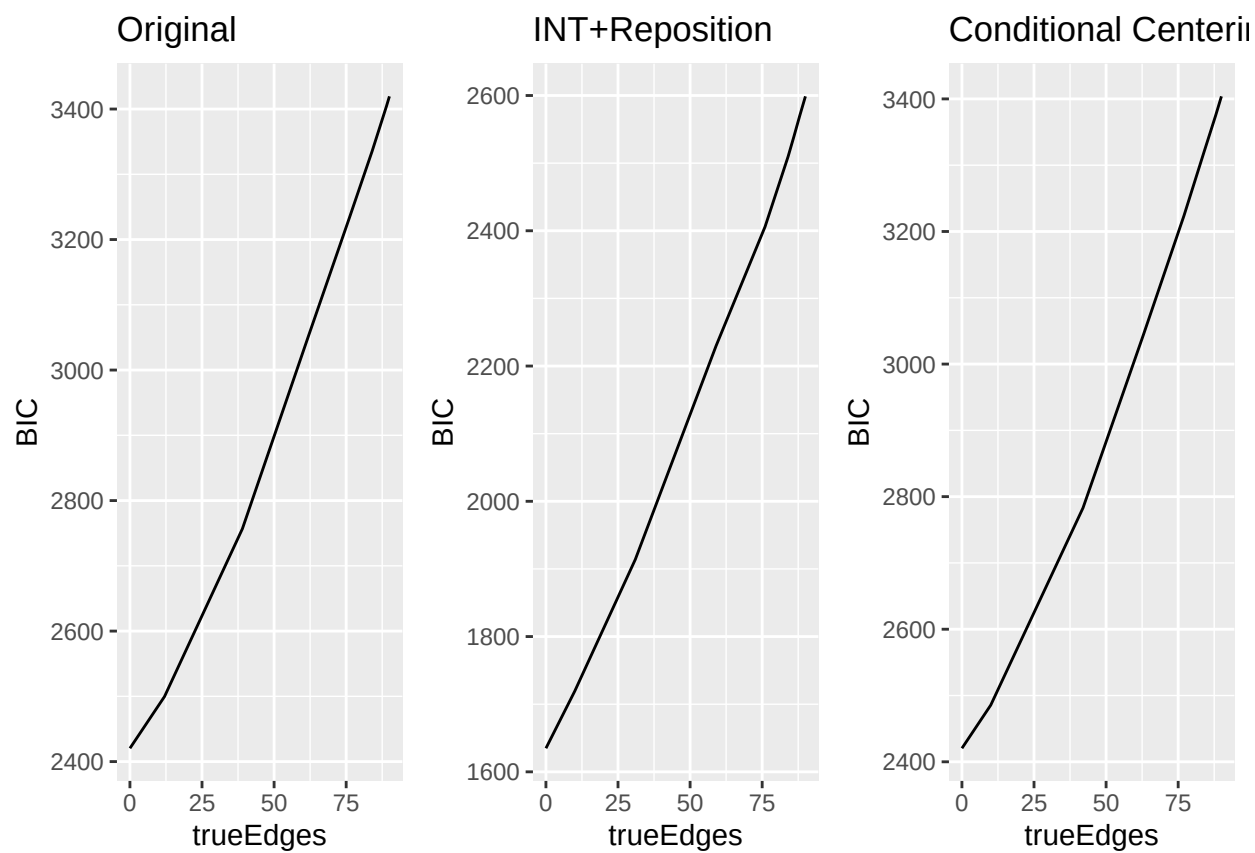
$\rho = 0.01$, Healthy



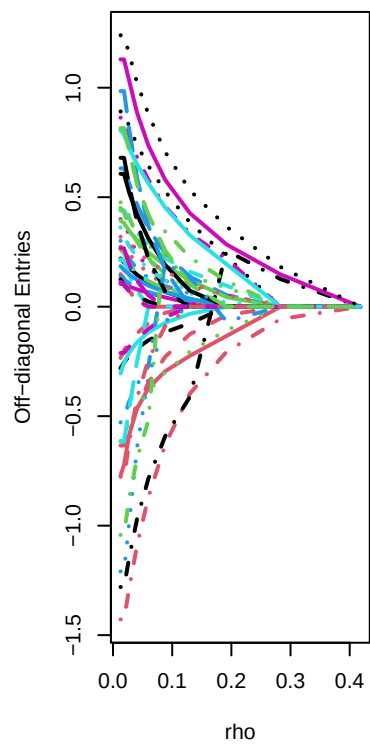
```
## [1] 0
```

```
hurdleComparison("Blood Cancer")
```

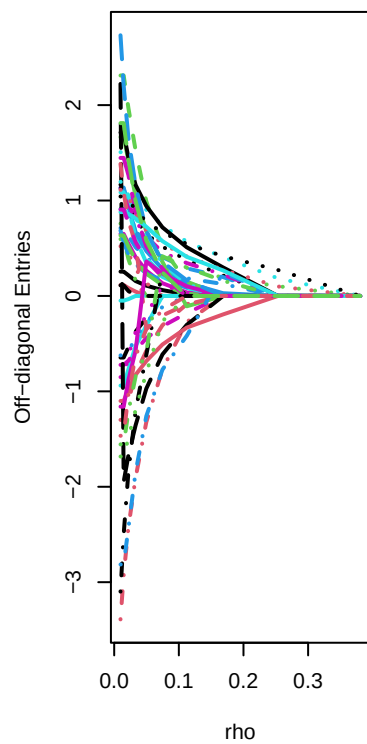
```
## Warning in solvePenProximal(theta, lambda[l], control, blocklist, hl, proxfun,  
## : Null step size
```



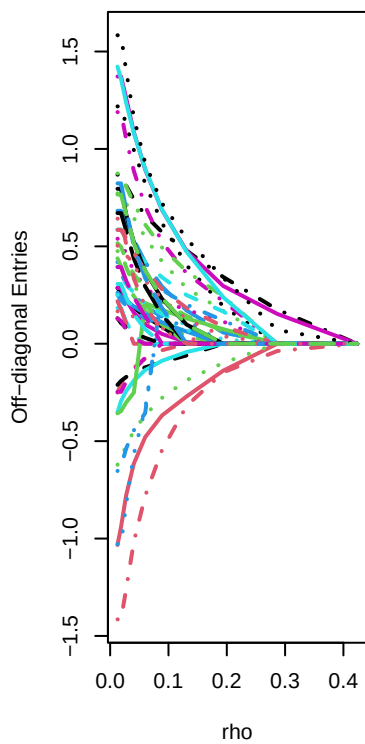
Coef Paths: Original



Coef Paths: INT+Reposition

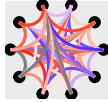


Coef Paths: Conditional Centeri



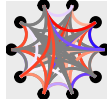
Original

$\rho = 0.01$, Healthy



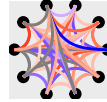
INT

$\rho = 0.01$, Healthy



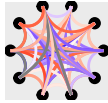
Conditional Cent

$\rho = 0.01$, Healthy



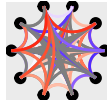
Original

$\rho = 0.03$, Healthy



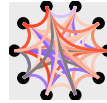
INT

$\rho = 0.02$, Healthy



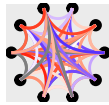
Conditional Cent

$\rho = 0.03$, Healthy



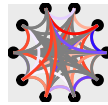
Original

$\rho = 0.01$, Healthy



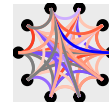
INT

$\rho = 0.01$, Healthy



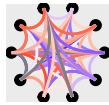
Conditional Cent

$\rho = 0.01$, Healthy



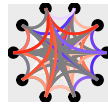
Original

$\rho = 0.03$, Healthy



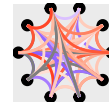
INT

$\rho = 0.02$, Healthy



Conditional Cent

$\rho = 0.03$, Healthy



```
## [1] 0
```

5. Do edge sets differ a lot between healthy and cancers?

```
allTypes <- unique(cancerTypes)
allTypes <- allTypes[allTypes != "Normal"]

## Preparing healthy normal data
healthy.original <- original.X[cancerTypes == "Normal", ]
healthy.INT <- INT(healthy.original, 0.25)

## Exact precision matrix
precMat.healthyOri <- solve(cov(healthy.original))
precMat.healthyINT <- solve(cov(healthy.INT))

## Take Uppper Triangular compoents
precCompo.healthyOri <- takeUpperTriangle(precMat.healthyOri)
precCompo.healthyINT <- takeUpperTriangle(precMat.healthyINT)
```

```
for (k in allTypes) {
  ## Prepare precision matrices for subset of data based on cancer types.
  X_k.original <- original.X[cancerTypes == k, ]
```

```

X_k.INT <- INT(X_k.original, 0.25)

precMat.original <- solve(cov(X_k.original))
precMat.INT <- solve(cov(X_k.INT))

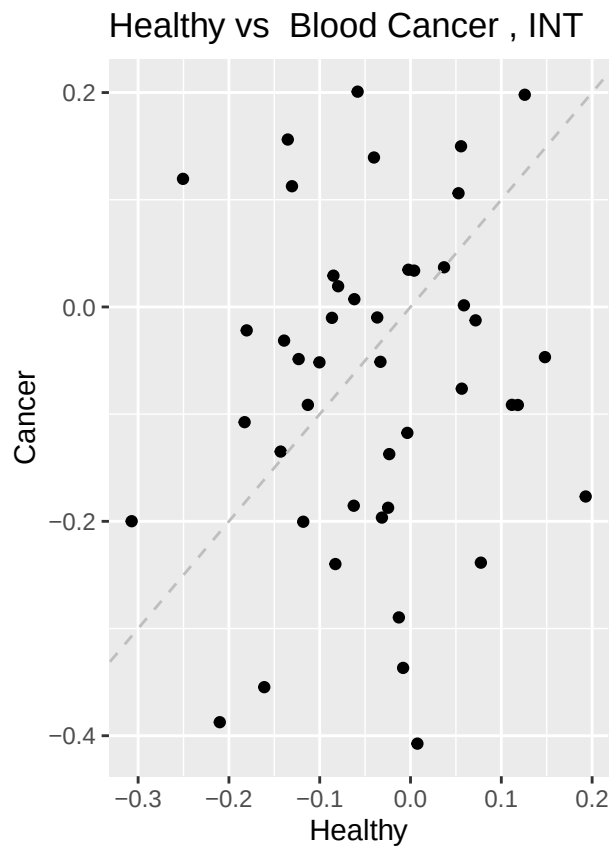
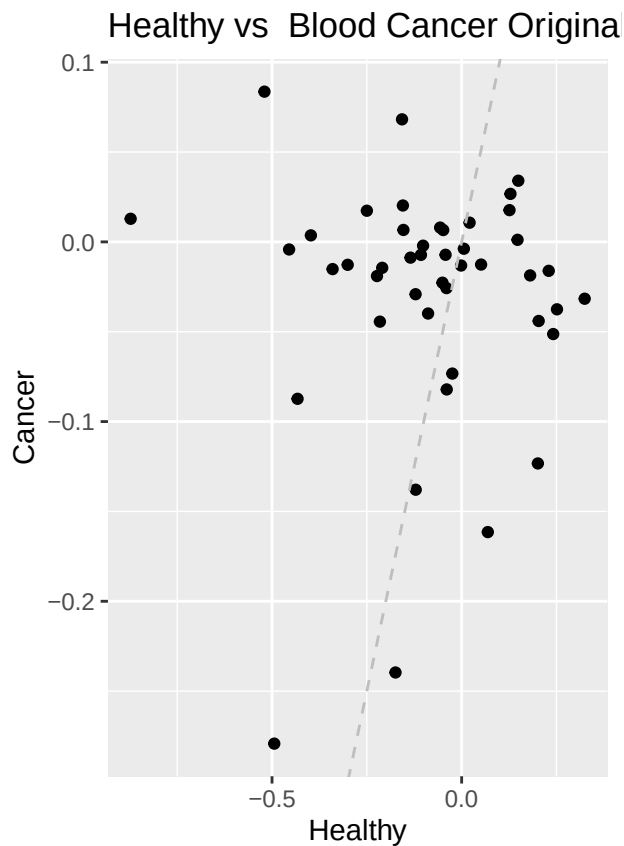
## Scatter Plot Comparison
precCompo.original <- takeUpperTriangle(precMat.original)
precCompo.INT <- takeUpperTriangle(precMat.INT)

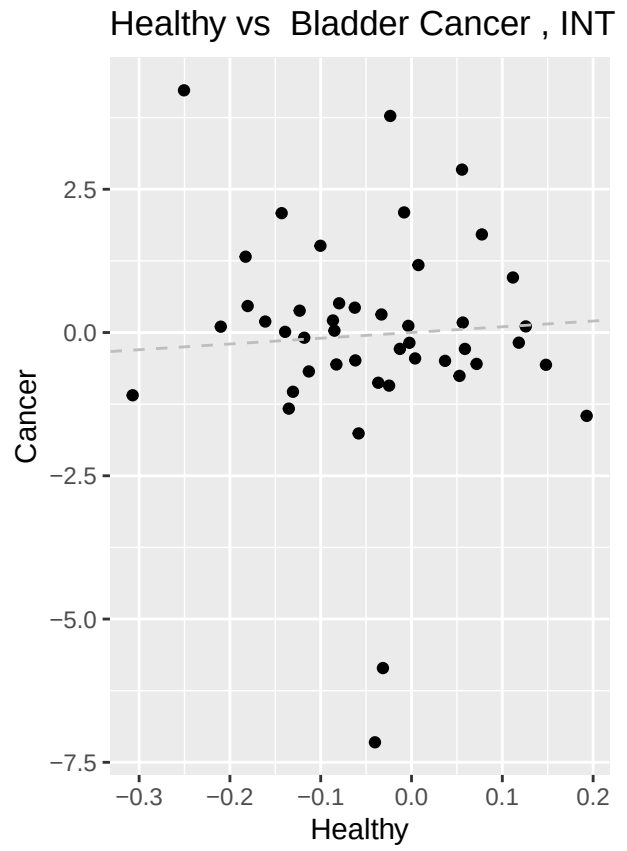
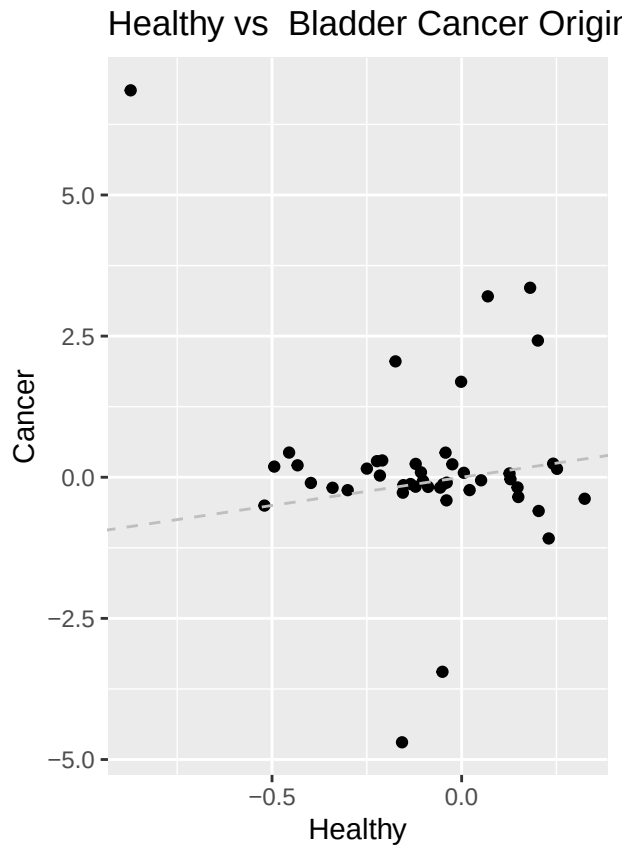
A <- ggplot(data = data.frame("Healthy" = precCompo.healthyOri, "Cancer" = precCompo.original)) +
  geom_point(aes(x = Healthy, y = Cancer)) +
  # coord_cartesian(xlim = c(-0.4,0.4), ylim = c(-0.4,0.4)) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey") +
  labs(title = paste("Healthy vs ", k, "Original"))

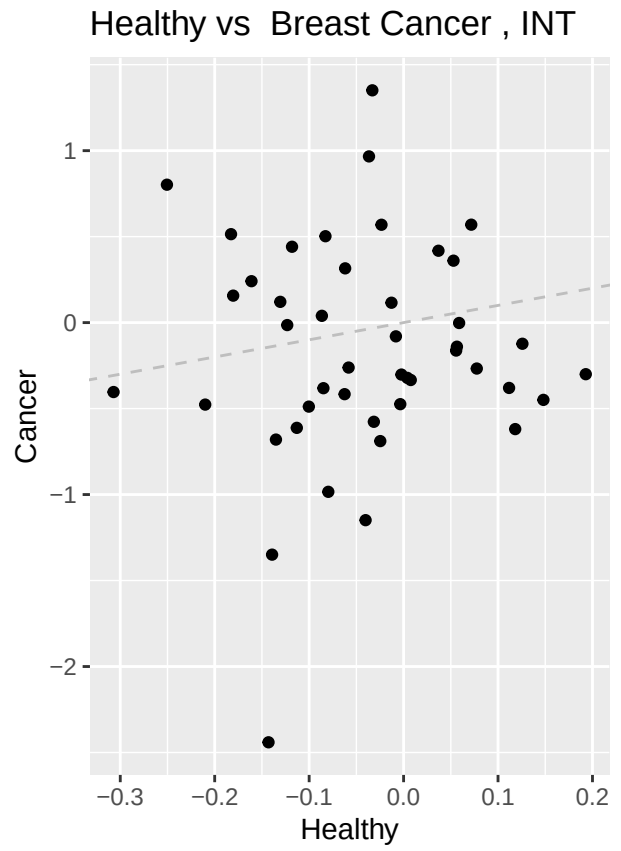
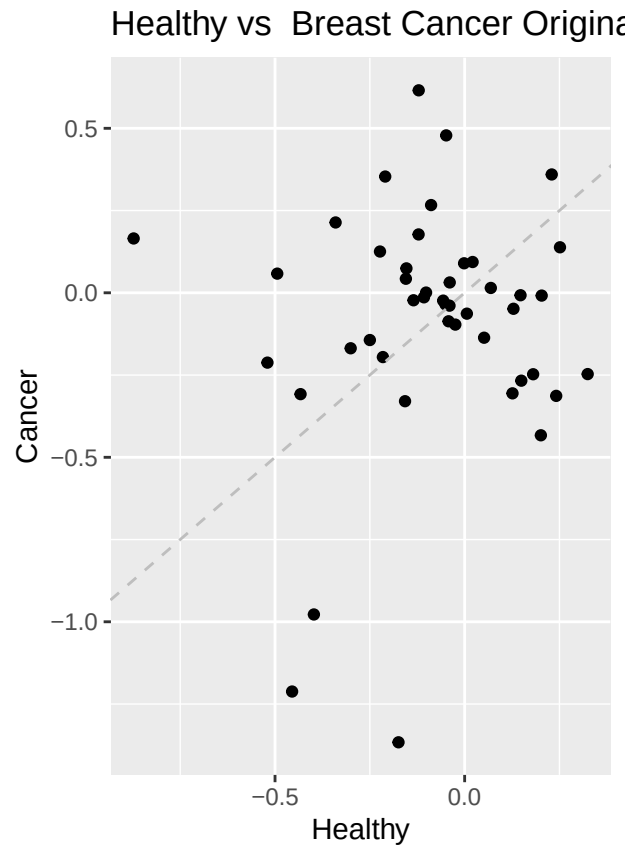
B <- ggplot(data = data.frame("Healthy" = precCompo.healthyINT, "Cancer" = precCompo.INT)) +
  geom_point(aes(x = Healthy, y = Cancer)) +
  # coord_cartesian(xlim = c(-0.4,0.4), ylim = c(-0.4,0.4)) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey") +
  labs(title = paste("Healthy vs ", k, ", INT"))

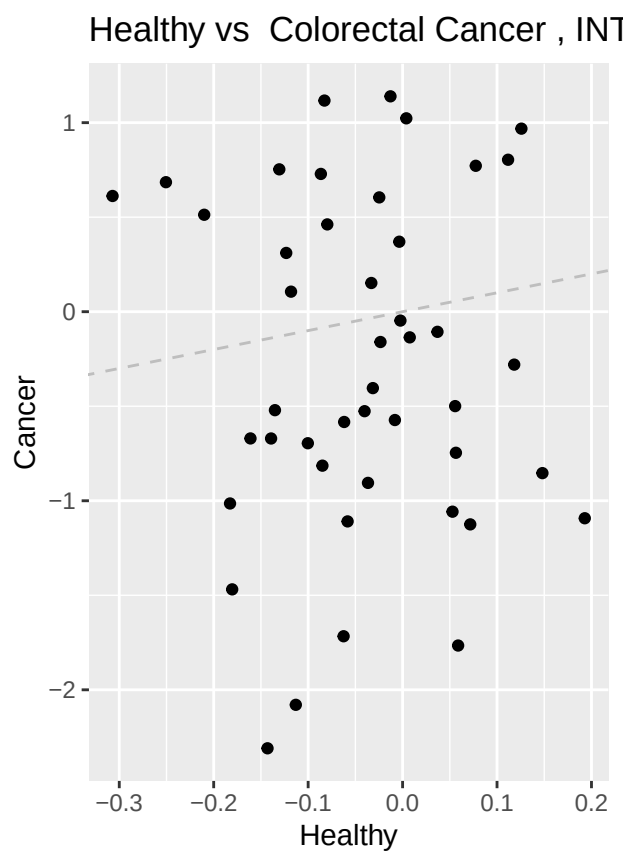
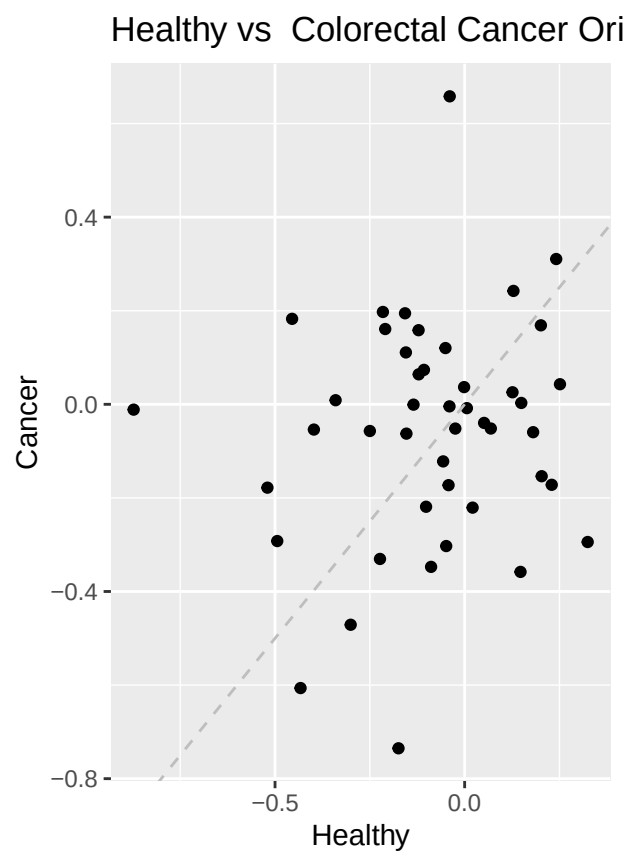
plot_grid(plotlist = list(A, B), nrow = 1, ncol = 2) %>% print()
}

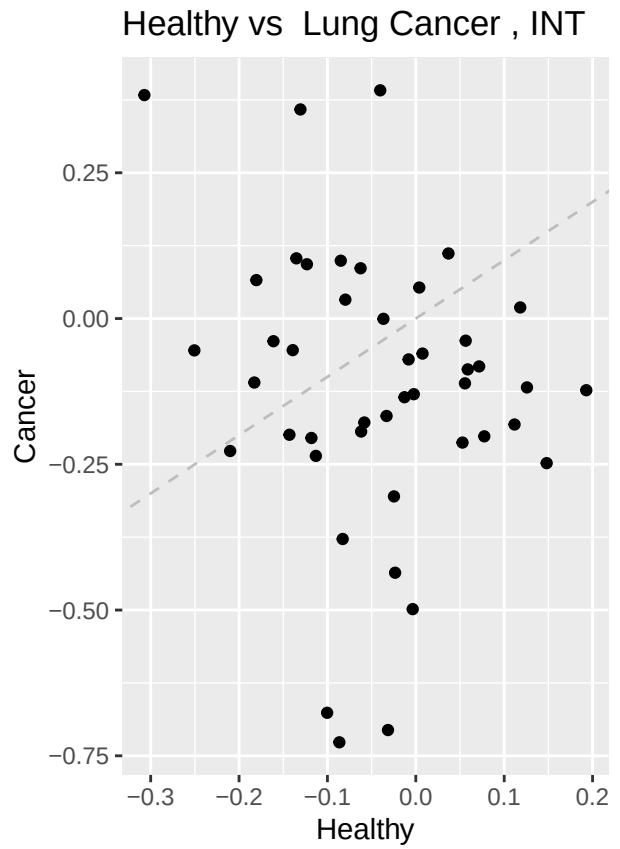
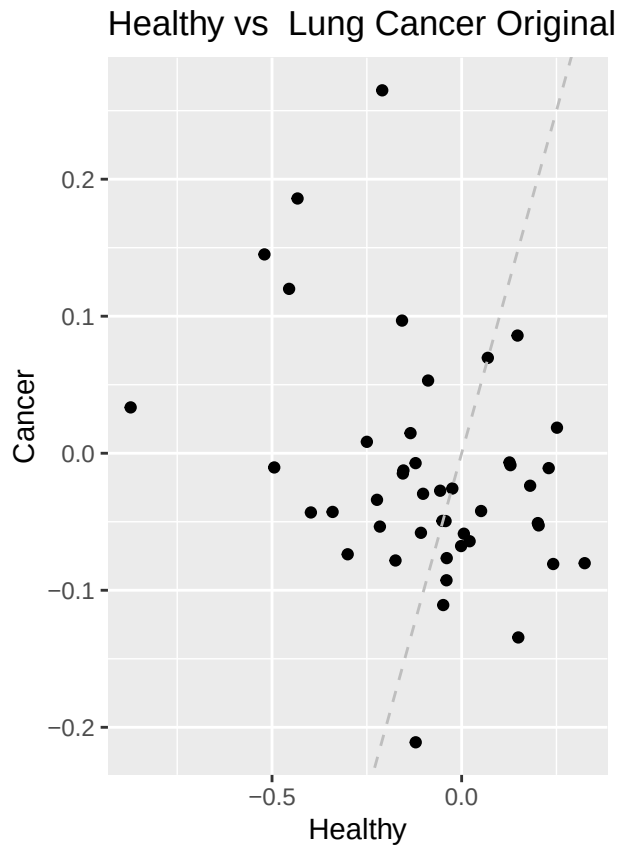
```

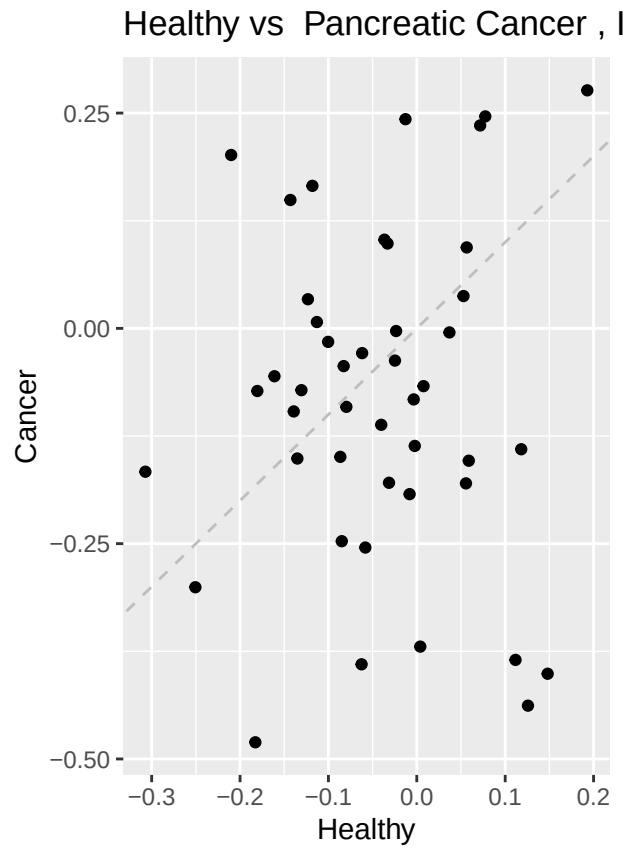
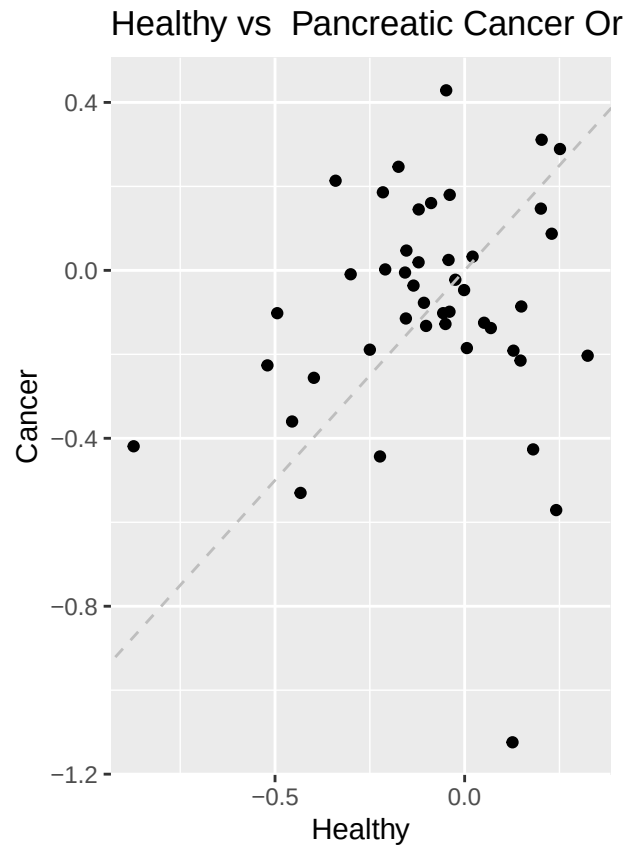


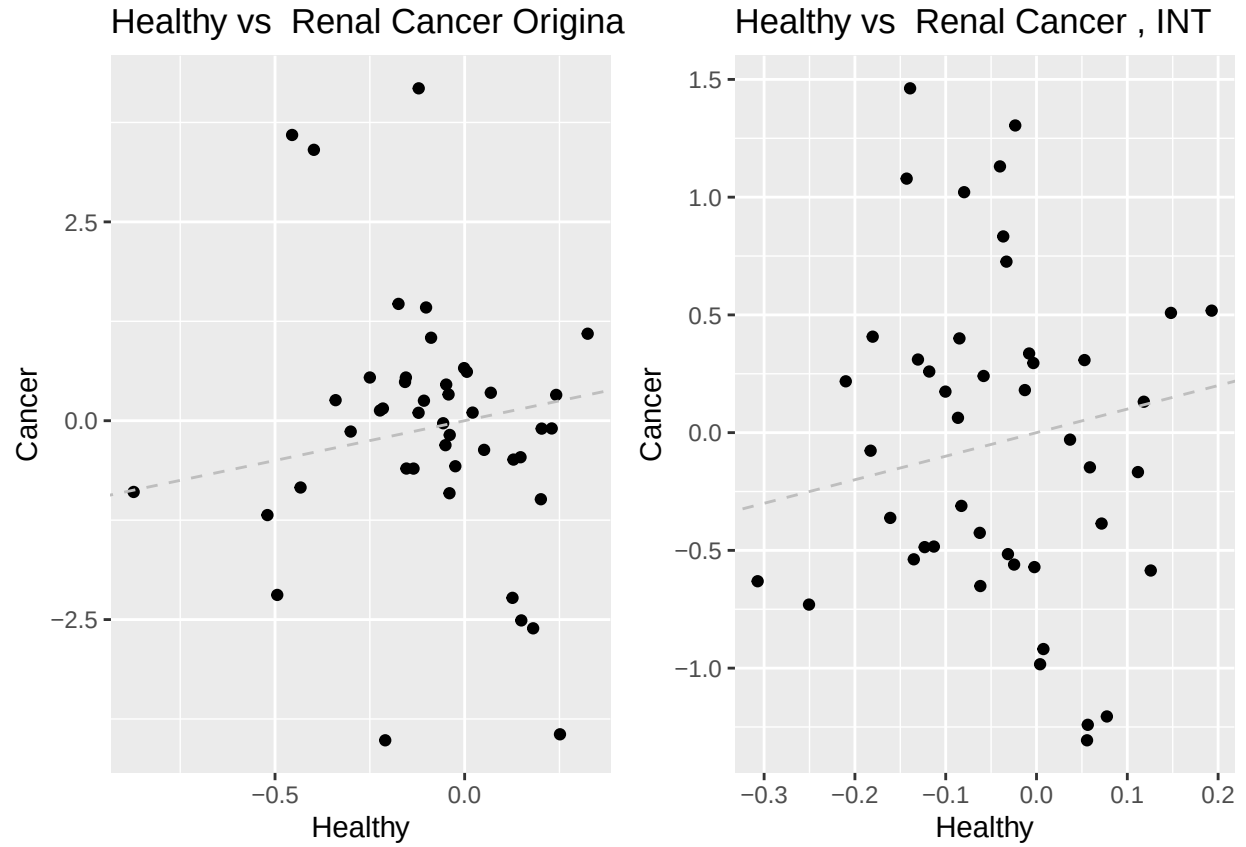












Observations: 1. Edge strengths of different cancers very different those of healthy normals. 2. Precision matrix components of Bladder cancer tend to concentrate around 0, with 3 outliers outside $[-2, 2]$ band. 3. Linear relationship in regular precision matrix entries were observed between healthy sample and colorectal, pancreatic, and renal cancer sample